



Hugging Face
Transformer & Data Library | As a Platform – HF Hub

Hugging Face : Transformer & Data Library | As a platform – HF Hub

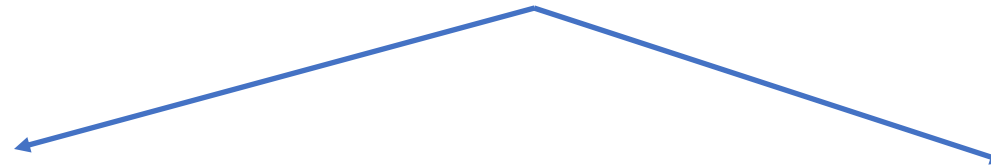
Transformer Library

- Support transformer architecture that can be used in NLP, CV, Speech Project.
- pypi.org;
transformers → from Hugging Face

Use with



Use hugging face transformers library for basically:



1. Applying state of the art transformer models for a variety of NLP, CV, Speech tasks.

2. Fine tuning pretrained models using your data sets. Or the Data sets provided by Hugging Face.

Datasets library

- Easily download and pre process, more than ~75,000 different data sets.

Hugging-Face Libraries facilitates for data handling, pre processing to modeling, training and post processing.

Hugging Face Transformer components

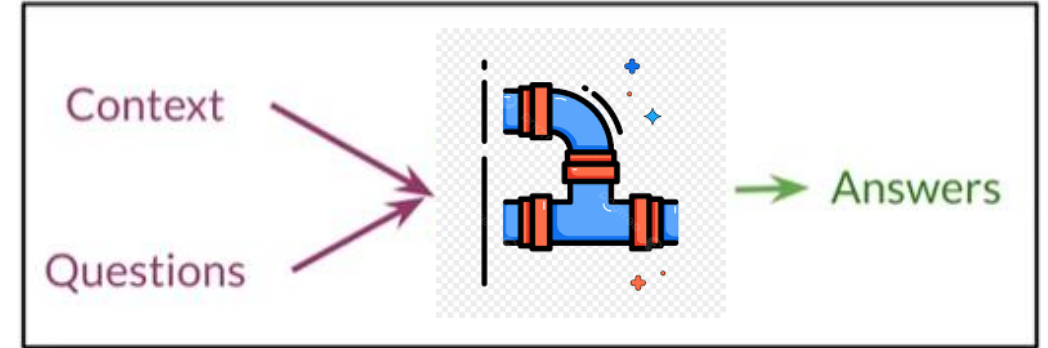
1. Pipeline

- The transformers library has the **pipeline** object that encapsulates everything need to run a model on any examples.
- With pipelines : can easily use state of the art transformer models for different NLP **tasks**.

Overall, Pipelines, take care of :

1. Pre-processing text inputs.
2. Running the model
3. Post processing the output to give human readable texts.

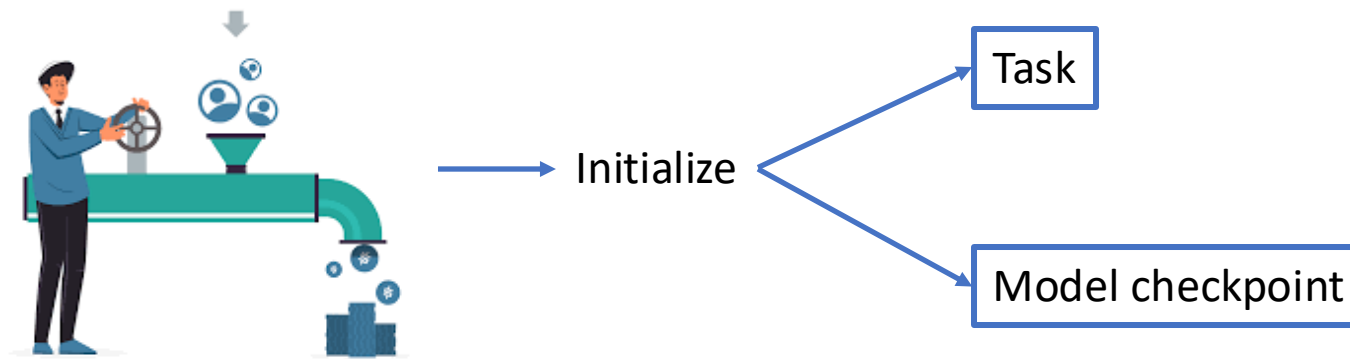
- Every pipeline has a default model so that one can use it without knowing what model to use.



- For instance, with a **pipeline for question answering task**, you can use a transformer to answer different questions, provided a context. You only need to specify the pipelines task, and provide a context with the questions you want to ask. The pipeline, takes care of everything else and gives you human readable answers.
- Can be used transformer models for virtually any NLP task with Hugging Face pipeline.

- Need to indicate **the task** that we want the pipeline to work on.
- Pipelines are objects that built for specific tasks and copy tasks to initialize a pipeline.

Pipeline



Use → Inputs for the task

Sentiment analysis : Sequence

Question Answering : Context and questions

2. Checkpoints

- A checkpoint **for any given model** is just the **set of parameters learned for a specific model**. That was trained with a particular dataset for some task like question answering.
- One can **fine tune a model from a starting checkpoints**. Using one of the more than 75K datasets available on Hugging Face or with your own data.
- Hugging face has a growing library with thousands of **pre-trained model checkpoints**. That you can use to fine tune the most popular transformer architectures.

Checkpoint : Set of learned parameters for a model using a training procedure for some task

Model checkpoint

Model

DistilBERT

Dataset

Stanford Question
Answering Dataset

Name in Hugging Face
(distilbert-base-cased-distilled-squad)

3. Tokenizer

There is a **Tokenizer** associated with every checkpoint to **pre-process the data**.

- The tokenizers do all the heavy lifting by translating the text into readable inputs for the model.

4. Model & Outputs

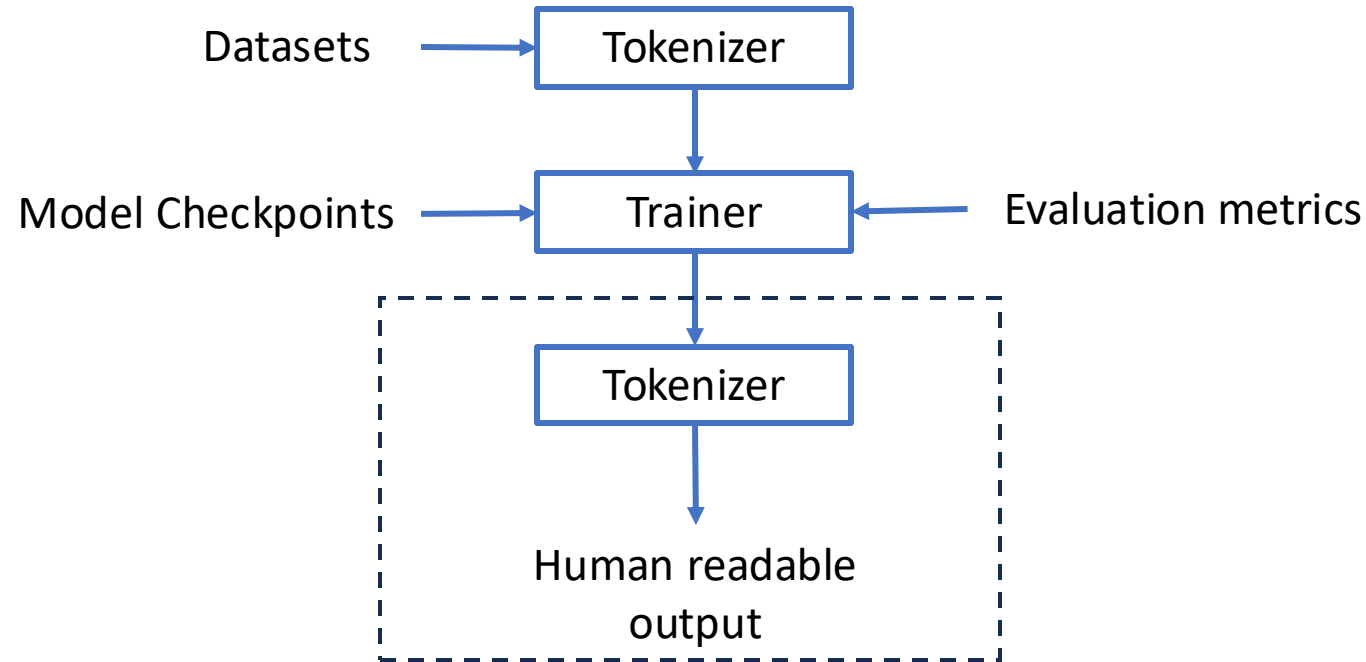
5. Trainer

- The trainer object lets us train the model without worrying about coding the training procedure.
- Through this, we can define parameters like: Number of epochs, Warm-ups steps, Weight decay , etc..
- For **PyTorch**, we can use the **trainer included in Hugging Face-transformers library**. Or create a custom one to train the model.
- For **TensorFlow**, it is recommended to use traditional Keras methods like '**Fits**'.

6. Metrics

- That seamlessly integrates into the training process to evaluate your model with pre-defined metrics like **BLEU, ROUGE**
- Hugging Face, also provides metrics like : [Glue](#).

All Together



- Choose any dataset, tokenize it, fine tune a model starting from a checkpoint (i.e. pre-trained model), get evaluated using pre-defined evaluation metrics, get an output - transform into a human readable text using tokenizer again .
- So, Hugging Face and the transformers library give you everything you need to fine tune a transformer.

Hugging Face : As a platform - Hugging Face hub

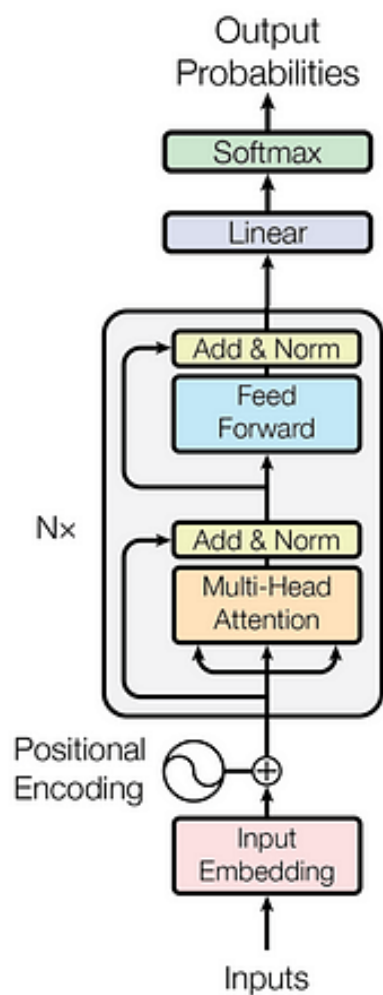
- **Hugging Face hub** - A platform to collaborate on machine learning models and datasets.
→ Models; DataSets; Spaces; Docs; etc
- **Model** - The model hub is host to more than 350K community contributed models, and it offers a way to select a model according → Task | Library | Trained on a specific Data set | Specific language | License.

Models can be from the hugging face transformers library but they're not limited to it.

Furthermore, it offer tools to push your own model - Host models and create own version of models work with data sets, existing ones/ your own.

Model card - Models can be further described by their model cards. It helps in explaining what this is about and give a little bit of information along with community members, code snippet examples . May play with the model using the hosted inference API .

- **Data Set** - The dataset hub is similar to the model hub hosting thousands of data sets, mostly community contributed. Can be selected → Task | Sizes | Specific language | License. These data sets also offer a comprehensive **data set card** explaining how the data set was designed, where the data came from and considerations when using the data. Additionally, it offers a quick link to models fine tuned or trained on these data sets.



BERT Base

BERT Large

**Number of
embedding
dimensions, d_{model}**

768

1024

**Number of encoder
blocks, N :**

12

24

**Number of attention
heads per encoder
block:**

12

16

**Size of hidden layer
in feedforward
network:**

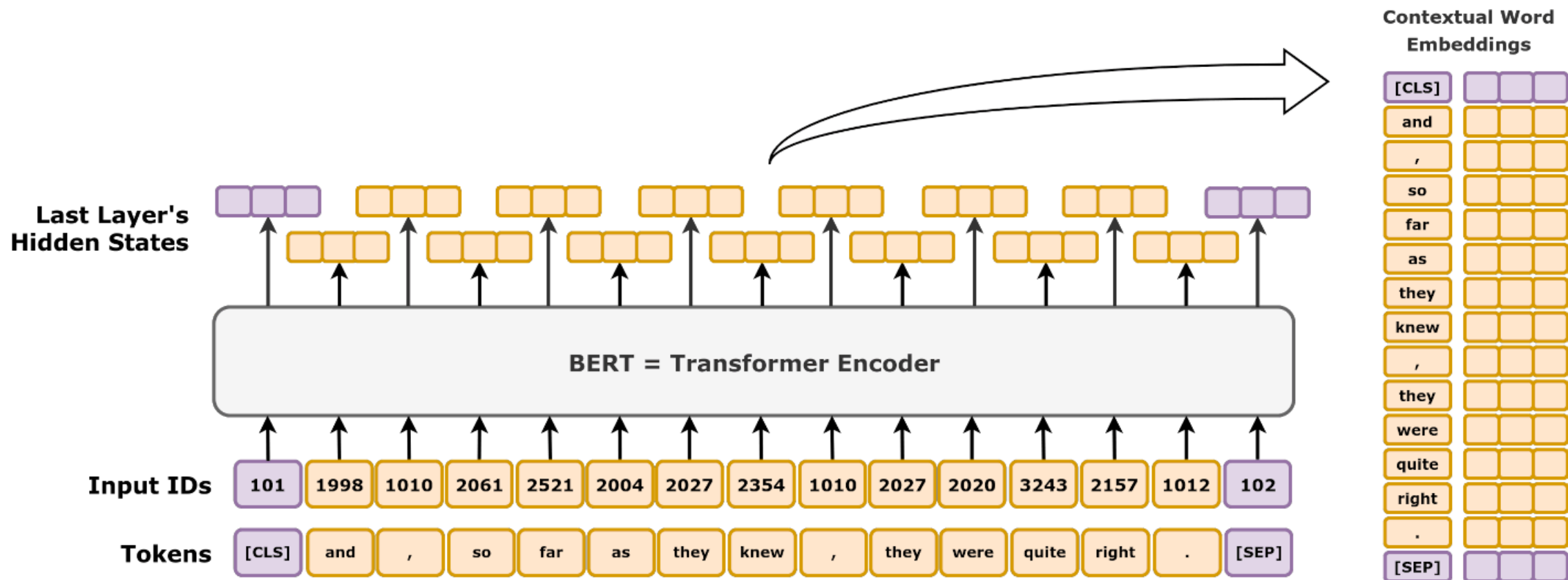
3072

4096

Total parameters:

110 million

340 million



Common Use Cases:

- **For sentence-level tasks** (like classification), you usually take the embedding of the special [CLS] token (from the last transformer layer).
 - Example: $X[:, 0, :]$, where X is the output of the final transformer layer and 0 is the index for the [CLS] token.
- **For token-level tasks** (like Named Entity Recognition), you typically use the entire sequence of token embeddings from the last transformer block (after contextualization).
 - Example: $X[:, :, :]$ from the last transformer layer.

GLUE Benchmark

[GLUE](#), the General Language Understanding Evaluation benchmark, is a collection of resources for training, evaluating, and analyzing natural language understanding systems.

GLUE is centered on [nine English sentence understanding tasks](#), which cover a broad range of domains, data quantities, and difficulties. As the goal of GLUE is to spur the development of generalizable NLU systems, the benchmark is designed such that good performance should require a model to share substantial knowledge across all tasks, while still maintaining some task-specific components.

The **Stanford Sentiment Treebank** is a corpus with fully labeled parse trees that allows for a complete analysis of the compositional effects of sentiment in language. The corpus is based on the dataset introduced by Pang and Lee (2005) and consists of 11,855 single sentences extracted from movie reviews. It was parsed with the Stanford parser and includes a total of 215,154 unique phrases from those parse trees, each annotated by 3 human judges.

Each phrase is labelled as either *negative*, *somewhat negative*, *neutral*, *somewhat positive* or *positive*. The corpus with all 5 labels is referred to as SST-5 or SST fine-grained. Binary classification experiments on full sentences (*negative* or *somewhat negative* vs *somewhat positive* or *positive* with *neutral* sentences discarded) refer to the dataset as SST-2 or SST binary.

Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: I think the food was okay.

Sentiment: ...

One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: I think the food was okay.

Sentiment:

Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral.**

Text: I think the food was great!

Sentiment: **Positive.**

Text: I think the food was horrible...

Sentiment: **Negative.**

Text: I think the food was okay.

Sentiment:

THANKS!